

Обработка ошибок. Results API

Лекция 8

Обработка исключений

Ошибки в приложении можно условно разделить на два типа: исключения, которые возникают в процессе выполнения кода (например, деление на 0), и стандартные ошибки протокола HTTP (например, ошибка 404).

Обычные исключения могут быть полезны для разработчика в процессе создания приложения, но простые пользователи не должны будут их видеть.

UseDeveloperExceptionPage

Для обработки исключений в приложении, которое находится в процессе разработки, предназначен специальный middleware -

DeveloperExceptionPageMiddleware. Однако вручную нам его не надо добавлять, поскольку оно добавляется автоматически. Так, если мы посмотрим на код класса WebApplicationBuilder, который применяется для создания приложения, то мы там можем увидеть там следующие строки:

```
if (context.HostingEnvironment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
}
```

Если приложение находится в состоянии разработки, то с помощью middleware `app.UseDeveloperExceptionPage()` приложение перехватывает исключения и выводит информацию о них разработчику.

Например, изменим код приложения следующим образом:

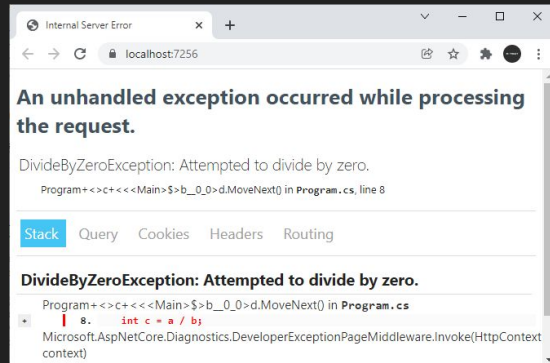
```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.UseDeveloperExceptionPage();

app.Run(async (context) =>
{
    int a = 5;
    int b = 0;
    int c = a / b;
    await context.Response.WriteAsync($"c = {c}");
});
```

```
app.Run();
```

В middleware в методе `app.Run()` симулируется генерация исключения путем деления на ноль. И если мы запустим проект, то в браузере мы увидим информацию об исключении:



UseDeveloperExceptionPage

Этой информации достаточно, чтобы определить где именно в коде произошло исключение.

Теперь посмотрим, как все это будет выглядеть для простого пользователя. Для этого изменим код приложения:

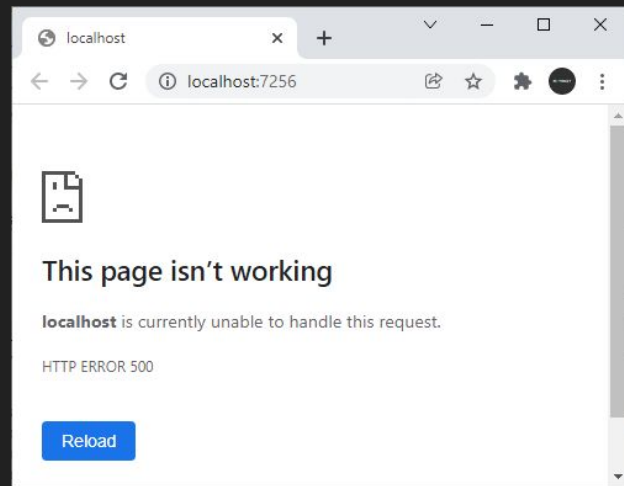
```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Environment.EnvironmentName = "Production"; // меняем имя окружения

app.Run(async (context) =>
{
    int a = 5;
    int b = 0;
    int c = a / b;
    await context.Response.WriteAsync($"c = {c}");
});

app.Run();
```

Выражение `app.Environment.EnvironmentName = "Production"` меняет имя окружения с "Development" (которое установлено по умолчанию) на "Production". В этом случае среда ASP.NET Core больше не будет расценивать приложение как находящееся в стадии разработки. Соответственно `middleware` из метода `app.UseDeveloperExceptionPage()` не будет перехватывать исключение. А приложение будет возвращать пользователю ошибку 500, которая указывает, что на сервере возникла ошибка при обработке запроса. Однако реально природа подобной ошибки может заключать в чем угодно.

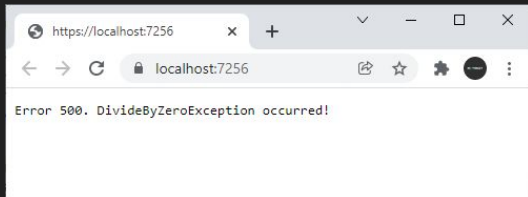


UseExceptionHandler

Это не самая лучшая ситуация, и нередко все-таки возникает необходимость дать пользователям некоторую информацию о том, что же все-таки произошло. Либо потребуется как-то обработать данную ситуацию. Для этих целей можно использовать еще один встроенный middleware - **ExceptionHandlerMiddleware**, который подключается с помощью метода **UseExceptionHandler()**. Он перенаправляет при возникновении исключения на некоторый адрес и позволяет обработать исключение. Например, изменим код приложения следующим образом:

Метод `app.UseExceptionHandler("/error");` перенаправляет при возникновении ошибки на адрес `"error"`.

Для обработки пути по определенному адресу здесь использовался метод `app.Map()`. В итоге при возникновении исключения будет срабатывать делегат из метода `app.Map()`, в котором пользователю будет отправляться некоторое сообщение со статусным кодом 500.



```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Environment.EnvironmentName = "Production"; // меняем имя окружения

// если приложение не находится в процессе разработки
// перенаправляем по адресу "/error"
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error");
}

// middleware, которое обрабатывает исключение
app.Map("/error", app => app.Run(async context =>
{
    context.Response.StatusCode = 500;
    await context.Response.WriteAsync("Error 500. DivideByZeroException occurred!");
}));

// middleware, где генерируется исключение
app.Run(async (context) =>
{
    int a = 5;
    int b = 0;
    int c = a / b;
    await context.Response.WriteAsync($"c = {c}");
});

app.Run();
```

UseExceptionHandler

Однако данный способ имеет небольшой недостаток - мы можем напрямую обратиться по адресу "/error" и получить тот же ответ от сервера. Но в этом случае мы можем использовать другую версию метода **UseExceptionHandler()**, которая принимает делегат, параметр которого представляет объект `IApplicationBuilder`

Следует учитывать, что `app.UseExceptionHandler()` следует помещать ближе к началу конвейера middleware.

```
var builder = WebApplication.CreateBuilder();
```

```
var app = builder.Build();
```

```
app.Environment.EnvironmentName = "Production";
```

```
if (!app.Environment.IsDevelopment())
```

```
{
```

```
    app.UseExceptionHandler(app => app.Run(async context =>
```

```
    {
```

```
        context.Response.StatusCode = 500;
```

```
        await context.Response.WriteAsync("Error 500. DivideByZeroException  
occurred!");
```

```
    }));
```

```
}
```

```
app.Run(async (context) =>
```

```
{
```

```
    int a = 5;
```

```
    int b = 0;
```

```
    int c = a / b;
```

```
    await context.Response.WriteAsync($"c = {c}");
```

```
});
```

```
app.Run();
```

Обработка ошибок HTTP

В отличие от исключений стандартный функционал проекта ASP.NET Core почти никак не обрабатывает ошибки HTTP, например, в случае если ресурс не найден. При обращении к несуществующему ресурсу мы увидим в браузере пустую страницу, и только через консоль веб-браузера мы сможем увидеть статусный код. Либо браузер может отобразить какую-то стандартную страницу.

Но с помощью компонента **StatusCodePagesMiddleware** можно добавить в проект отправку информации о статусном коде. Для этого применяется метод **app.UseStatusCodePages()**:

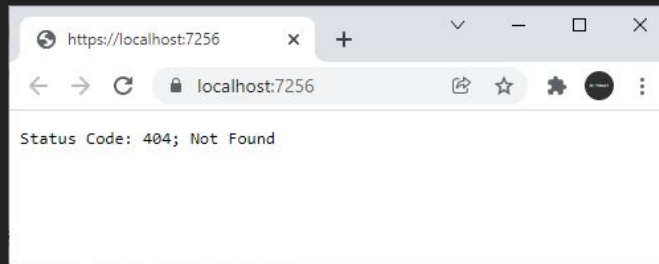
```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

// обработка ошибок HTTP
app.UseStatusCodePages();

app.Map("/hello", () => "Hello ASP.NET Core");

app.Run();
```

Здесь мы можем обращаться только по адресу `"/hello"`. При обращении ко всем остальным адресам браузер отобразит базовую информацию об ошибке:



Метод **UseStatusCodePages()** следует вызывать ближе к началу конвейера обработки запроса, в частности, до добавления middleware для работы со статическими файлами и до добавления конечных точек.

Настройка сообщения

Сообщение, отправляемое методом `UseStatusCodePages()` по умолчанию, не очень информативное. Однако одна из версий метода позволяет настроить отправляемое пользователю сообщение. В частности, мы можем изменить вызов метода так:

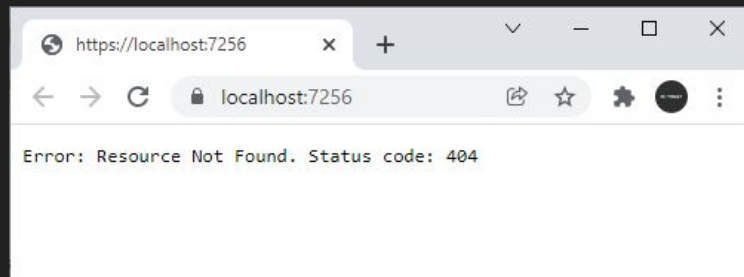
```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

// обработка ошибок HTTP
app.UseStatusCodePages("text/plain", "Error: Resource Not Found. Status code: {0}");

app.Map("/hello", () => "Hello ASP.NET Core");

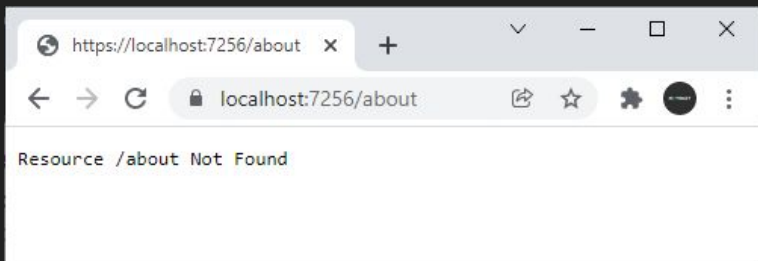
app.Run();
```

В качестве первого параметра указывается MIME-тип ответа - в данном случае простой текст ("text/plain"). В качестве второго параметра передается собственно то сообщение, которое увидит пользователь. В сообщении мы можем передать код ошибки через плейсхолдер "{0}".



Установка обработчика ошибок

Еще одна версия метода **UseStatusCodePages()** позволяет более детально задать обработчик ошибок. В частности, она принимает делегат, параметр которого - объект **StatusCodeContext**. В свою очередь, объект **StatusCodeContext** имеет свойство **HttpContext**, из которого мы можем получить всю информацию о запросе и настроить отправку ответа. Например:



```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

// обработка ошибок HTTP
app.UseStatusCodePages(async statusCodeContext =>
{
    var response = statusCodeContext.HttpContext.Response;
    var path = statusCodeContext.HttpContext.Request.Path;

    response.ContentType = "text/plain; charset=UTF-8";
    if (response.StatusCode == 403)
    {
        await response.WriteAsync($"Path: {path}. Access Denied ");
    }
    else if (response.StatusCode == 404)
    {
        await response.WriteAsync($"Resource {path} Not Found");
    }
});

app.Map("/hello", () => "Hello ASP.NET Core");

app.Run();
```

Методы UseStatusCodePagesWithRedirects и UseStatusCodePagesWithReExecute

Вместо метода UseStatusCodePages() мы также можем использовать еще пару других, которые также обрабатывают ошибки HTTP.

С помощью метода app.UseStatusCodePagesWithRedirects() можно выполнить переадресацию на определенный метод, который непосредственно обработает статусный код:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.UseStatusCodePagesWithRedirects("/error/{0}");

app.Map("/hello", () => "Hello ASP.NET Core");
app.Map("/error/{statusCode}", (int statusCode) => $"Error. Status Code: {statusCode}");

app.Run();
```

Здесь будет идти перенаправление по адресу "/error/{0}". В качестве параметра через плейсхолдер "{0}" будет передаваться статусный код ошибки.

Но теперь при обращении к несуществующему ресурсу клиент получит статусный код 302 / Found. То есть формально несуществующий ресурс будет существовать, просто статусный код 302 будет указывать, что ресурс перемещен на другое место - по пути "/error/404".

Подобное поведение может быть неудобно, особенно с точки зрения поисковой индексации, и в этом случае мы можем применить другой метод app.UseStatusCodePagesWithReExecute():

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.UseStatusCodePagesWithReExecute("/error/{0}");

app.Map("/hello", () => "Hello ASP.NET Core");
app.Map("/error/{statusCode}", (int statusCode) =>
    $"Error. Status Code: {statusCode}");

app.Run();
```

В качестве параметра метод UseStatusCodePagesWithReExecute() принимает путь к ресурсу, который будет обрабатывать ошибку. И также с помощью плейсхолдера {0} можно передать статусный код ошибки. То есть в данном случае при возникновении ошибки будет вызываться конечная точка

```
app.Map("/error/{statusCode}", (int statusCode) =>
    $"Error. Status Code: {statusCode}");
```

Формально мы получим тот же ответ, так как так же будет идти перенаправление на путь "/error/404". Но теперь браузер получит оригинальный статусный код 404.

Методы UseStatusCodePagesWithRedirects и UseStatusCodePagesWithReExecute

Также можно задействовать другую версию метода, которая в качестве второго параметра принимает параметры строки запроса

```
app.UseStatusCodePagesWithReExecute("/error", "?code={0}");
```

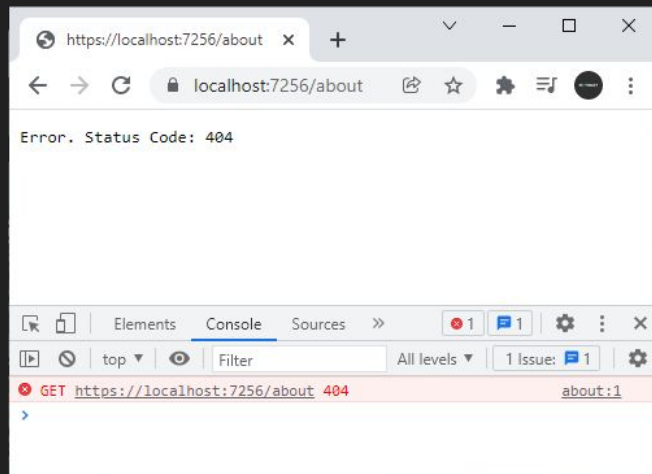
Первый параметр метода указывает на путь перенаправления, а второй задает параметры строки запроса, которые будут передаваться при перенаправлении. Вместо плейсхолдера {0} опять же будет передаваться статусный код ошибки. Пример использования:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.UseStatusCodePagesWithReExecute("/error", "?code={0}");

app.Map("/hello", () => "Hello ASP.NET Core");
app.Map("/error", (string code) => $"Error Code: {code}");

app.Run();
```



Results API. Введение в Results API

Для упрощения отправки ответа ASP.NET Core предоставляет специальный API - **Results API**. Он реализуется посредством методов статического класса **Results**:

- **Accepted()**: отправляет клиенту статусный код 202, который указывает, что запрос принят к обработке. Может принимать два необязательных параметра: первый параметр представляет адрес для отслеживания состояния запроса, а второй - произвольный объект, отправляемый в ответ клиенту
- **AcceptedAtRoute()**: также отправляет клиенту статусный код 202. Может принимать три необязательных параметра: первый параметр представляет имя маршрута, а второй - параметры для этого маршрута, третий - произвольный объект, отправляемый в ответ клиенту
- **BadRequest()**: отправляет клиенту статусный код 400, который указывает, что запрос не может быть обработан в силу содержащихся в нем ошибок. Может принимать необязательный параметр - произвольный объект, отправляемый в ответ клиенту
- **Bytes()**: отправляет в ответ клиенту массив байтов, обычно применяется для отправки файлов
Этот метод по своему действию аналогичен методу `File(Byte[], String, String, Boolean, Nullable<DateTimeOffset>, EntityTagHeaderValue)`
- **Challenge()**: создает объект `IResult`, для которого выполняется метод `ChallengeAsync(HttpContext, String, AuthenticationProperties)`
Поведение этого метода зависит от сервиса `IAuthenticationService`. В качестве возможных ответов могут статусные коды 401 (не авторизован) и 403 (доступ запрещен)
- **Conflict()**: отправляет ответ со статусным кодом 409
- **Content()**: отправляет в ответе некоторую строку
- **Created()**: отправляет ответ со статусным кодом 201
- **CreatedAtRoute()**: отправляет ответ со статусным кодом 201
- **File()**: отправляет в ответ файл
- **Forbid()**: создает объект `IResult`, для которого выполняется метод `ForbidAsync(HttpContext, String, AuthenticationProperties)`. По умолчанию отправляет статусный код 403 (доступ запрещен)
- **Json()**: отправляет ответ в формате JSON
- **LocalRedirect()**: перенаправляет на один из локальных адресов
- **NoContent()**: отправляет статусный код 204
- **NotFound()**: отправляет ответ со статусным кодом 404
- **Ok()**: отправляет ответ со статусным кодом 200
- **Problem()**: применяет формат [ProblemDetails](#) для отправки ответа.
- **Redirect()**: перенаправляет на определенный адрес
- **RedirectToRoute()**: перенаправляет на определенный маршрут
- **SignIn()**: создает объект `IResult`, для которого выполняется метод `SignInAsync(HttpContext, String, ClaimsPrincipal, AuthenticationProperties)`
- **SignOut()**: создает объект `IResult`, для которого выполняется метод `SignOutAsync(HttpContext, String, AuthenticationProperties)`
- **StatusCode()**: отправляет определенный статусный код
- **Stream()**: пишет в ответ поток данных. По своему действию аналогичен методу `File(Stream, String, String, Nullable<DateTimeOffset>, EntityTagHeaderValue, Boolean)`
- **Text()**: отправляет в ответ некоторое текстовое содержимое. Аналогичен методу `Content(String, String, Encoding)`
- **Unauthorized()**: отправляет статусный код 401
- **UnprocessableEntity()**: отправляет ответ со статусным кодом 422
- **ValidationProblem()**: отправляет ответ со статусным кодом 400

Введение в Results API

Применим один из методов:

```
var builder = WebApplication.CreateBuilder();  
var app = builder.Build();
```

```
app.Map("/hello", () => Results.Text("Hello  
ASP.NET Core"));  
app.Map("/", () => "Hello ASP.NET Core");
```

```
app.Run();
```

В данном случае в приложении определены две конечные точки: одна обрабатывает запросы по маршруту `/hello`, а другая - запросы к корню веб-приложения. Первая конечная точка для отправки ответа использует метод `Results.Text()`. Другая конечная точка напрямую возвращает некоторую строку. Однако в реальности обе конечные точки продуцируют почти аналогичный ответ: также будет отправляться одна и та же строка со статусным кодом 200. Разница будет заключаться только в отдельных заголовках (в частности, метод `Results.Text` добавляет заголовок `"Content-Length"`).

Подобным образом можно обрабатывать запрос в отдельном методе, который возвращает объект `IResult`:

```
var builder = WebApplication.CreateBuilder();  
var app = builder.Build();
```

```
app.Map("/hello", SendHello);  
app.Map("/", () => "Hello ASP.NET Core 6");
```

```
app.Run();
```

```
IResult SendHello()  
{  
    return Results.Text("Hello ASP.NET Core");  
}
```

Хотя мы можем отправлять некоторый ответ напрямую из конечных точек, не используя методы `Results`, тем не менее методы `Results API` могут быть полезны, когда необходимо установить какие-то дополнительные параметры ответа, например, кодировку, форматирование в json. И конечно мы можем использовать методы `Results API` для отправки в конечных точках статусных кодов, файлов или выполнения редиректов.

Хотя `Results API` в большей степени предназначено для использования в конечных точках `ASP.NET Core`, те же методы могут применяться для отправки ответа и в обычных компонентах `middleware`. Например:

```
var builder = WebApplication.CreateBuilder();  
var app = builder.Build();  
  
app.Run(async context =>  
{  
    await Results.Text("Hello ASP.NET Core").ExecuteAsync(context);  
});  
  
app.Run();
```

Отправка текста и метод Content

Метод **Content()** отправляет текстовое содержимое и позволяет при этом задать тип содержимого и кодировку. Одна из версий метода:

```
public static IActionResult Content (string content, string? contentType = default, System.Text.Encoding? contentEncoding = default);
```

- Параметр **content** задает текстовое содержимое ответа
- Параметр **contentType** задает MIME-тип ответа
- Параметр **contentEncoding** задает кодировку

```
var builder = WebApplication.CreateBuilder();  
var app = builder.Build();
```

```
app.Map("/", () => Results.Content("你好", "text/plain", System.Text.Encoding.Unicode));
```

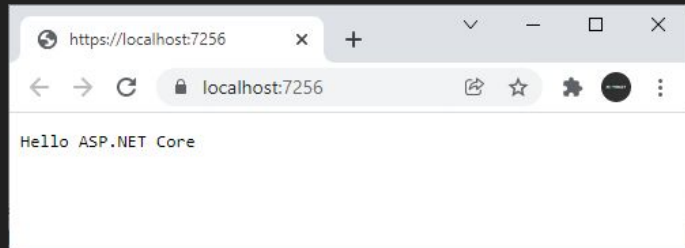
```
app.Run();
```

Если указан только первый параметр, то метод по умолчанию будет использовать в качестве типа содержимого "text/plain", а в качестве кодировки "utf-8"

```
var builder = WebApplication.CreateBuilder();  
var app = builder.Build();
```

```
app.Map("/", () => Results.Content("Hello ASP.NET Core"));
```

```
app.Run();
```



Text

Метод **Text()** работает аналогичным образом, он также отправляет текст и принимает те же параметры:

```
public static IActionResult Text (string content, string? contentType = default, System.Text.Encoding?
contentTypeEncoding = default);
```

Пример работы:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Map("/chinese", () => Results.Text("你好", "text/plain", System.Text.Encoding.Unicode));
app.Map("/", () => Results.Text("Hello World"));

app.Run();
```

Отправка json

Для отправки данных в формате json применяется метод **Results.Json()**:

```
public static IResult Json (object? data, JsonSerializerOptions? options = default, string? contentType = default, int? statusCode = default);
```

Параметры метода:

- **data**: отправляемый объект
- **options**: объект `System.Text.Json.JsonSerializerOptions?`, который задает параметры сериализации
- **contentType**: тип содержимого в виде строки. Если этот параметр не указан, то по умолчанию применяется тип `"application/json; charset=utf-8"`
- **statusCode**: отправляемый вместе с json код статуса. Если этот параметр не указан, то по умолчанию код статус - 200

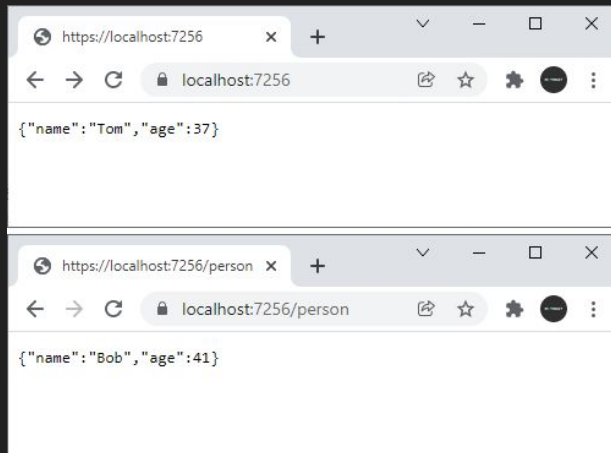
Применение метода:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Map("/person", () => Results.Json(new Person("Bob", 41))); // отправка объекта Person
app.Map("/", () => Results.Json(new { name = "Tom", age = 37 })); // отправка анонимного объекта

app.Run();

record class Person(string Name, int Age);
```



Отправка json

Если надо конкретизировать параметры сериализации в json, то можно использовать второй параметр метода, который представляет тип `System.Text.Json.JsonSerializerOptions?`:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Map("/sam", () => Results.Json(new Person("Sam", 25),
    new()
    {
        PropertyNameCaseInsensitive = false,
        NumberHandling =
System.Text.Json.Serialization.JsonNumberHandling.WriteAsString
    }));

app.Map("/bob", () => Results.Json(new Person("Bob", 41),
    new(System.Text.Json.JsonSerializerDefaults.Web)));

app.Map("/tom", () => Results.Json(new Person("Tom", 37),
    new(System.Text.Json.JsonSerializerDefaults.General)));

app.Run();

record class Person(string Name, int Age);
```

Нередко формат json также применяется и для отправки ошибок. В этом случае мы можем задать статусный код ошибки с помощью последнего параметра:

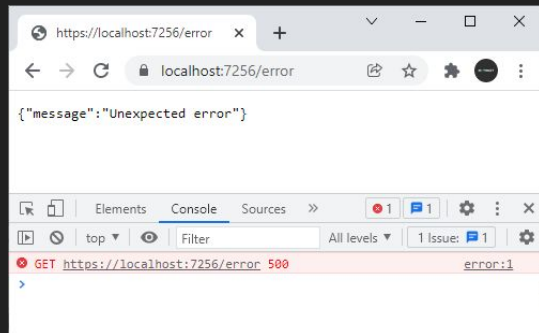
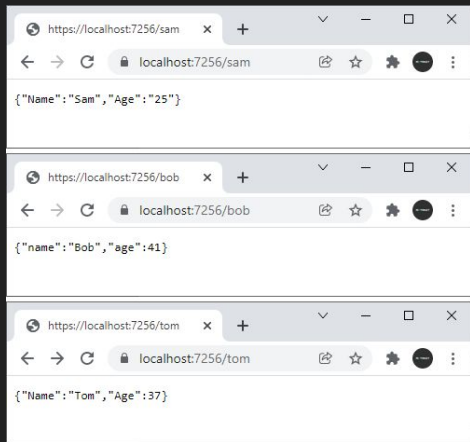
```
var builder =
WebApplication.CreateBuilder();
var app = builder.Build();

app.Map("/error", () => Results.Json(new
{message="Unexpected error"}, statusCode:
500));

app.Map("/", () => "Hello World");

app.Run();
```

В данном случае посылаем объект с информацией об ошибке и статусный код 500:



Переадресация в Results API. LocalRedirect

Для переадресации на локальный адрес в рамках приложения применяется метод **LocalRedirect()**:

```
public static IResult LocalRedirect (string localUrl, bool permanent = false, bool preserveMethod = false);
```

Параметры метода:

- **localUrl**: локальный адрес для переадресации
- **permanent**: указывает, будет ли переадресация постоянной (отправляется статусный код 301) или временной (отправляется статусный код 302).
- **preserveMethod**: если равен true, то сохраняется оригинальный метод HTTP-запроса.

Применение метода:

```
var builder = WebApplication.CreateBuilder();  
var app = builder.Build();  
  
app.Map("/old", () => Results.LocalRedirect("/new"));  
app.Map("/new", () => "New Address");  
  
app.Map("/", () => "Hello World");  
  
app.Run();
```

В данном случае при обращении по пути `"/old"` будет осуществляться редирект на адрес `"/new"`. Поскольку второй параметр не задан, то будет выполняться временная переадресация.

Метод Redirect

Метод **Redirect()** также осуществляет переадресацию и принимает те же параметры, что и `LocalRedirect()`, только адрес для переадресации может не только локальным, но и внешним:

```
public static IActionResult Redirect (string url, bool permanent = false, bool preserveMethod = false);
```

Пример работы:

```
var builder = WebApplication.CreateBuilder();  
var app = builder.Build();  
  
app.Map("/old", () => Results.Redirect("https://8xbyte.dev"));  
app.Map("/", () => "Hello World");  
  
app.Run();
```

RedirectToRoute

Метод **RedirectToRoute()** выполняет переадресацию на определенный маршрут:

```
public static IActionResult RedirectToRoute (string? routeName = default, object?  
    routeValues = default, bool permanent = false, bool preserveMethod = false, string? fragment =  
    default);
```

Параметры метода:

- **routeName**: название маршрута
- **routeValues**: значения для параметров маршрута
- **permanent**: указывает, будет ли переадресация постоянной (отправляется статусный код 301) или временной (отправляется статусный код 302).
- **preserveMethod**: если равен true, то сохраняется оригинальный метод HTTP-запроса.
- **fragment**: фрагмент, который добавляется к адресу для переадресации

Отправка статусных кодов в Results API. StatusCode

Нередко возникает необходимость отправить в ответ на запрос какой-либо статусный код. Например, если пользователь пытается получить доступ к ресурсу, который недоступен или для которого у пользователя нету прав. Либо просто необходимо уведомить пользователя с помощью статусного кода об успешном выполнении операции. И для этого Results API предоставляет ряд методов.

Метод **StatusCode()** позволяет отправить любой статусный код, числовой код которого передается в метод в качестве параметра:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Map("/about", () => Results.StatusCode(401));
app.Map("/", () => "Hello World");

app.Run();
```

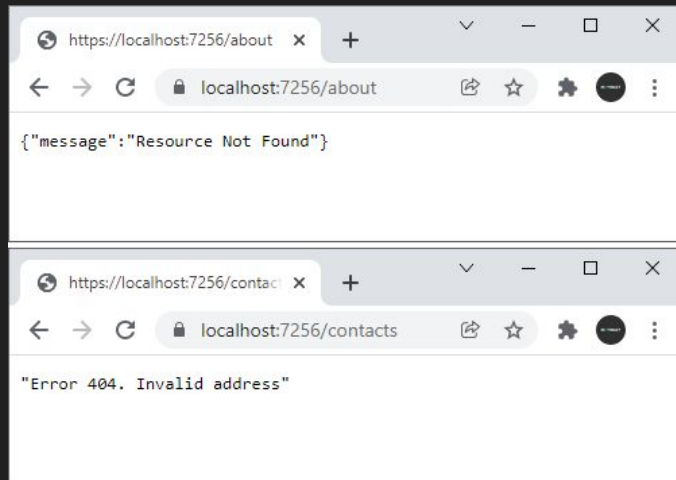
Подобным образом мы можем послать клиенту любой другой статусный код. Но для некоторых отдельных статусных кодов имеются свои отдельные методы.

Метод NotFound

Метод **NotFound()** посылает код 404, уведомляя клиента о том, что ресурс не найден. В качестве параметра в метод можно передать некоторый объект для отправки клиенту, например, сообщение об ошибке:

```
var builder = WebApplication.CreateBuilder();  
var app = builder.Build();  
  
app.Map("/about", () => Results.NotFound(new { message= "Resource Not Found"}));  
app.Map("/contacts", () => Results.NotFound("Error 404. Invalid address"));  
app.Map("/", () => "Hello World");  
  
app.Run();
```

Стоит отметить, что сложные объекты по умолчанию сериализуются в формат json.



Unauthorized

Метод **Unauthorized()** посылает код 401, уведомляя пользователя, что он не авторизован для доступа к ресурсу:

```
var builder = WebApplication.CreateBuilder();  
var app = builder.Build();  
  
app.Map("/contacts", () => Results.Unauthorized());  
app.Map("/", () => "Hello World");  
  
app.Run();
```

BadRequest

Метод **BadRequest()** посылает код 400, который говорит о том, что запрос некорректный. В качестве параметра можно передать некоторый объект для отправки клиенту:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

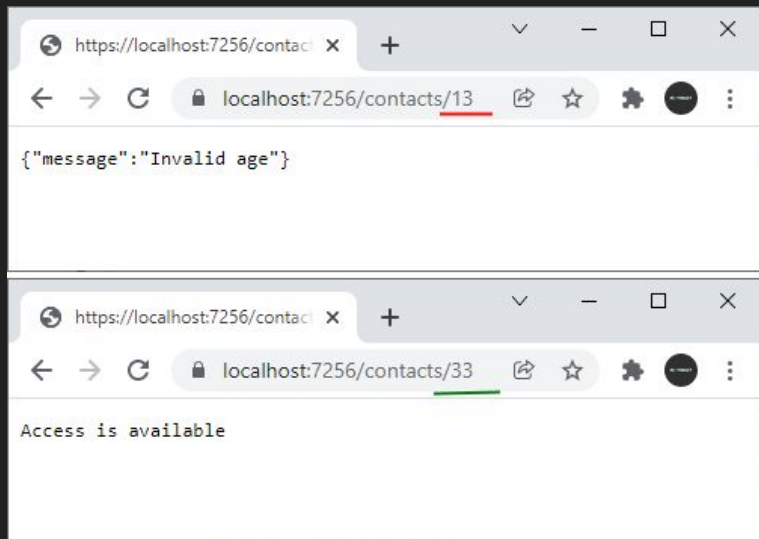
app.Map("/contacts/{age:int}", (int age) =>
{
    if (age < 18)
        return Results.BadRequest(new { message = "Invalid age" });
    else
        return Results.Content("Access is available");
});

app.Map("/", () => "Hello World");

app.Run();
```

В данном случае если параметр маршрута `age` меньше 18, то посылает клиенту статусный код 400 с сообщением "Invalid age". Иначе посылает некоторое стандартное сообщение

Причем опять же посылаемый объект по умолчанию сериализуется в формат json.



Ok

Метод **Ok()** посылает статусный код 200, уведомляя об успешном выполнении запроса. В качестве параметра метод принимает отправляемую информацию:

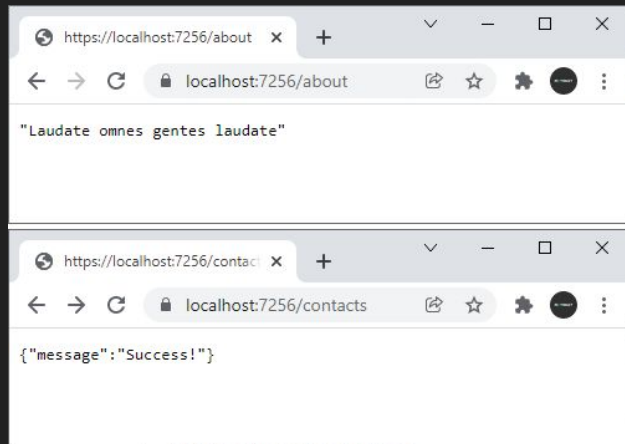
```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Map("/about", () => Results.Ok("Laudate omnes gentes
laudate"));

app.Map("/contacts", () => Results.Ok(new { message =
"Success!" }));

app.Map("/", () => "Hello World");

app.Run();
```



Отправка файлов в Results API. Отправка файла как массива байтов

Для отправки файлов с помощью Results API может применяться метод `File()`, который имеет ряд версий.

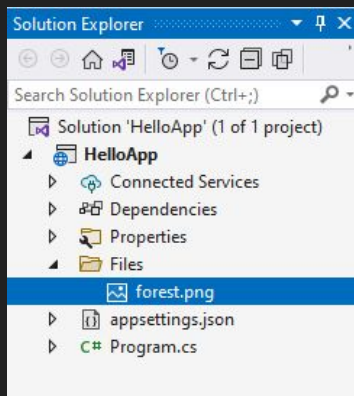
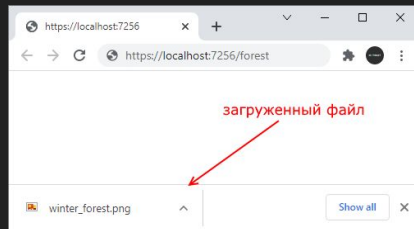
Для отправки файла как массива байтов применяется следующая версия метода:

```
public static IResult File (byte[] fileContents, string? contentType = default,
string? fileDownloadName = default, bool enableRangeProcessing = false,
DateTimeOffset? lastModified = default,
Microsoft.Net.Http.Headers.EntityTagHeaderValue? entityTag = default);
```

Параметры метода:

- **fileContents**: содержимое файла в виде массива байтов
- **contentType**: тип содержимого файла - заголовок Content-Type
- **fileDownloadName**: имя файла для загрузки
- **enableRangeProcessing**: значение типа bool. Если оно равно true, то допустима обработка данных по частям
- **lastModified**: значение типа Nullable<DateTimeOffset>, которое указывает на дату последнего изменения файла
- **entityTag**: значение типа EntityTagHeaderValue, которое ассоциировано с файлом

Например, создадим в проекте новую папку **Files**, в которую добавим какой-нибудь файл. Например, в моем случае это файл **forest.png**:



Для отправки файла определим следующий код:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();
```

```
app.Map("/forest", async () =>
{
```

```
    string path = "Files/forest.png";
```

```
    byte[] fileContent = await
```

```
File.ReadAllBytesAsync(path); // считываем файл в
массив байтов
```

```
    string contentType = "image/png"; //
```

```
установка mime-типа
```

```
    string downloadName = "winter_forest.png"; //
```

```
установка загружаемого имени
```

```
    return Results.File(fileContent, contentType,
downloadName);
```

```
});
```

```
app.Map("/", () => "Hello World");
```

```
app.Run();
```

И при обращении по пути `/forest` пользователю будет отправлен файл `forest.png` под именем `winter_forest.png`:

Отправка в виде файлового потока

Еще одна версия метода `File()` позволяет отправить данные в виде потока `Stream`:

```
public static IResult File (System.IO.Stream fileStream, string? contentType = default,
    string? fileDownloadName = default, DateTimeOffset? lastModified = default,
    Microsoft.Net.Http.Headers.EntityTagHeaderValue? entityTag = default, bool enableRangeProcessing = false);
```

Здесь меняется только первый параметр, который в данной версии представляет содержимое файла в виде объекта `Stream`. Остальные параметры остаются те же, что и в предыдущей версии

Также, как и в предыдущем примере, отправим файл `forest.png` из папки `Files`:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Map("/forest", () =>
{
    string path = "Files/forest.png";
    FileStream fileStream = new FileStream(path, FileMode.Open);
    string contentType = "image/png";
    string downloadName = "winter_forest.png";
    return Results.File(fileStream, contentType, downloadName);
});

app.Map("/", () => "Hello World");

app.Run();
```

Отправка файла по определенному пути

Если у нас есть путь к файлу, то проще использовать третью версию метода `File()`, которая в качестве первого параметра принимает путь к файлу:

```
public static IResult File (string path, string? contentType = default,
string? fileDownloadName = default, DateTimeOffset? lastModified = default,
Microsoft.Net.Http.Headers.EntityTagHeaderValue? entityTag = default, bool
enableRangeProcessing = false);
```

Остальные параметры те же, что в ранее рассмотренных версиях метода. Однако стоит отметить, что данная версия метода вычисляет пути к файлам относительно папки, которая задается параметром `WebRootPath` - по умолчанию это папка `wwwroot`. То есть файл для отправки по умолчанию должен располагаться в этой папке

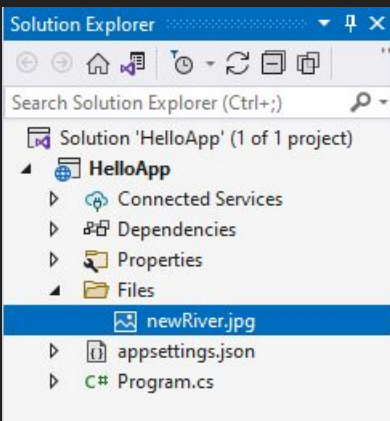
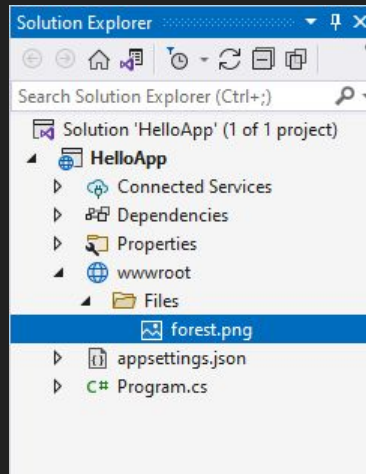
Используем эту версию метода для отправки файла `wwwroot/Files/forest.png`:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Map("/forest", () =>
{
    string path = "Files/forest.png";
    string contentType = "image/png";
    string downloadName = "winter_forest.png";
    return Results.File(path, contentType, downloadName);
});

app.Map("/", () => "Hello World");

app.Run();
```



Но это поведение мы можем переопределить. Допустим, в проекте есть папка `Files`. И мы хотим, чтобы приложение автоматически подхватывало из нее файлы.

Для этого добавим новый каталог для статических файлов:

```
var builder =
WebApplication.CreateBuilder(
    new WebApplicationOptions {
        WebRootPath = "Files" }); //
добавляем папку для хранения файлов
var app = builder.Build();

app.Map("/river", () =>
{
    string path = "newRiver.jpg";
    string contentType = "image/jpeg";
    string downloadName = "river.jpg";
    return Results.File(path,
contentType, downloadName);
});
```

```
app.Map("/", () => "Hello World");
```

```
app.Run();
```

В данном случае путь к файлу `newRiver.jpg` будет рассчитываться относительно папки `Files`.

Определение своего типа IActionResult

Встроенные типы Result API покрывают различные ситуации. Однако встроенных типов может оказаться недостаточно. И в этом случае мы можем определить свой тип IActionResult.

Например, по умолчанию Results API не предоставляет типа для отправки кода html. Рассмотрим, как мы можем сделать IActionResult для отправки html.

Для этого определим следующий код приложения

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

// отправляем html-код при обращении по пути "/"
app.Map("/", () => Results.Extensions.Html(@<!DOCTYPE html>
<html>
<head>
<title>8xbyte</title>
<meta charset='utf-8' />
</head>
<body>
<h1>Hello World</h1>
</body>
</html>
"));

app.Run();

class HtmlResult: IActionResult
{
    string htmlCode = "";
    public HtmlResult(string htmlCode) => this.htmlCode = htmlCode;

    public async Task ExecuteAsync(HttpContext context)
    {
        context.Response.ContentType = "text/html; charset=utf-8";
        await context.Response.WriteAsync(htmlCode);
    }
}

static class ResultsHtmlExtension
{
    public static IActionResult Html(this IActionResultExtensions ext, string htmlCode) => new HtmlResult(htmlCode);
}
```

Определение своего типа IActionResult

Ключевым моментом здесь является класс **HtmlResult**, который собственно и отправляет html-код:

```
class HtmlResult: IActionResult
{
    string htmlCode = "";
    public HtmlResult(string htmlCode) => this.htmlCode = htmlCode;

    public async Task ExecuteAsync(HttpContext context)
    {
        context.Response.ContentType = "text/html; charset=utf-8";
        await context.Response.WriteAsync(htmlCode);
    }
}
```

Он должен реализовать интерфейс **IActionResult**, который определяет метод **ExecuteAsync()**

Через конструктор класса получаем отправляемый код html. А в методе `ExecuteAsync()` через параметр типа `HttpContext` устанавливаем заголовок и отправляем html-код.

Для создания данного объекта можно определить метод расширения для типа `IResultExtensions`:

```
static class ResultsHtmlExtension
{
    public static IActionResult Html(this IActionResultExtensions ext, string htmlCode) => new HtmlResult(htmlCode);
}
```

Затем мы можем применить данный метод для отправки ответа при обращении к какой-нибудь конечной точке:

```
app.Map("/", () => Results.Extensions.Html(@"<!DOCTYPE html>..."));
```

Хотя в принципе также можно было бы напрямую создать объект `HtmlResult`:

```
app.Map("/", () => new HtmlResult(@"<!DOCTYPE html>..."));
```

Но в любом случае при обращении по адресу "/" приложение отправит html-код, который отобразит браузер.